

```
//CREATING AND DISPLAY CODE FOR DOUBLY LINKED LIST|
```

```
# include<stdio.h>
# include<stdlib.h>
struct node
{
    struct node *prev;
    int data;
    struct node *next;
};
struct node *head=0,*temp=0,*tail=0;
void create()
{
    struct node *newnode=0;
    int i,n;
    printf("\nenter n size: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        newnode=(struct node *)malloc(sizeof(struct node));
        printf("\nenter node %d data: ",i+1);
        scanf("%d",&newnode->data);
        newnode->next=0;
        newnode->prev=0;
    }
}
```

```
    if(head==NULL)
    {
        head=tail=newnode;
    }
    else
    {
        tail->next=newnode;
        newnode->prev=tail;
        tail=newnode;
    }
}

void display()
{
    temp=head;
    while(temp!=0)
    {
        printf(" %d",temp->data);
        temp=temp->next;
    }
}

int main()
{
    create();
    display();
    return 0;
}
```

```
enter n size: 4
```

```
enter node 1 data: 10
```

```
enter node 2 data: 20
```

```
enter node 3 data: 30
```

```
enter node 4 data: 40
```

```
10 20 30 40
```

//INSERTING NODE AT FRONT IN DOUBLY LINKED LIST

```
# include<stdio.h>
# include<stdlib.h>
struct node
{
    struct node *prev;
    int data;
    struct node *next;
};
struct node *head=0,*temp=0,*tail=0;
void create()
{
    struct node *newnode=0;
    int i,n;
    printf("\nenter n size: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        newnode=(struct node *)malloc(sizeof(struct node));
        printf("\nenter node %d data: ",i+1);
        scanf("%d",&newnode->data);
        newnode->prev=NULL;
        newnode->next=NULL;
    }
}
```

```
newnode->next=NULL;
if(head==NULL)
{
    head=tail=newnode;
}
else
{
    tail->next=newnode;
    newnode->prev=tail;
    tail=newnode;
}
}

void display()
{
    temp=head;
    while(temp!=0)
    {
        printf(" %d",temp->data);
        temp=temp->next;
    }
}
```

```
int main()
{
    create();
    display();
    struct node *enode=0;
    enode=(struct node *)malloc(sizeof(struct node));
    printf("\nenter data for newly created node: ");
    scanf("%d",&enode->data);
    enode->prev=NULL;
    enode->next=NULL;
    head->prev=enode;
    enode->next=head;
    head=enode;
    printf("\nAFTER INSERTING NODE AT FRONT: ");
    display();
    return 0;
}
```

enter n size: 4

enter node 1 data: 10

enter node 2 data: 20

enter node 3 data: 30

enter node 4 data: 40

10 20 30 40

enter data for newly created node: 50

AFTER INSERTING NODE AT FRONT: 50 10 20 30 40


```

//INSERTING NODE AT END IN DOUBLY LINKED LIST
# include<stdio.h>
# include<stdlib.h>
struct node
{
    struct node *prev;
    int data;
    struct node *next;
};
struct node *head=0,*temp=0,*tail=0;
void create()
{
    struct node *newnode=0;
    int i,n;
    printf("\nenter n size: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        newnode=(struct node *)malloc(sizeof(struct node));
        printf("\nenter node %d data: ",i+1);
        scanf("%d",&newnode->data);
        newnode->next=0;
        newnode->prev=0;
    }
}

```



```
        if(head==NULL)
        {
            head=tail=newnode;
        }
        else
        {
            tail->next=newnode;
            newnode->prev=tail;
            tail=newnode;
        }
    }
}

void display()
{
    temp=head;
    while(temp!=0)
    {
        printf(" %d",temp->data);
        temp=temp->next;
    }
}
```

```
int main()  
{  
    create();  
    display();  
    struct node *enode=NULL;  
    enode=(struct node *)malloc(sizeof(struct node));  
    printf("\nEnter data for newly created node: ");  
    scanf("%d",&enode->data);  
    enode->next=0;  
    enode->prev=0;  
    tail->next=enode;  
    enode->prev=tail;  
    tail=enode;  
    printf("\nAFTER INSERTING NODE AT END POSITION: ");  
    display();  
    return 0;  
}
```

```
enter n size: 4
```

```
enter node 1 data: 10
```

```
enter node 2 data: 20
```

```
enter node 3 data: 30
```

```
enter node 4 data: 40
```

```
10 20 30 40
```

```
enter data for newly created node: 50
```

```
AFTER INSERTING NODE AT END POSITION: 10 20 30 40 50
```

//INSERTING NODE AT PARTICULAR POSITION IN DOUBLY LINKED LIST

```
# include<stdio.h>
# include<stdlib.h>
struct node
{
    struct node *prev;
    int data;
    struct node *next;
};
struct node *head=NULL, *tail=NULL, *temp=NULL;
void create()
{
    struct node *newnode=NULL;
    int i,n;
    printf("\nenter n size: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        newnode=(struct node *)malloc(sizeof(struct node));
        printf("\nenter node %d data: ",i+1);
        scanf("%d",&newnode->data);
        newnode->prev=0;
        newnode->next=0;
    }
}
```

```
    if(head==NULL)
    {
        head=newnode;
        tail=newnode;
    }
    else
    {
        tail->next=newnode;
        newnode->prev=tail;
        tail=newnode;
    }
}

void display()
{
    temp=head;
    while(temp!=NULL)
    {
        printf(" %d",temp->data);
        temp=temp->next;
    }
}
```

```
int main()
{
    create();
    display();
    struct node *enode=NULL;
    int pos,i;
    enode=(struct node *)malloc(sizeof(struct node));
    printf("\nenter newly creates node data: ");
    scanf("%d",&enode->data);
    enode->prev=0;enode->next=0;
    printf("\nenter position: ");
    scanf("%d",&pos);
    if(pos==1)
    {
        enode->next=head;
        head->prev=enode;
        head=enode;
    }
}
```

else

```
{  
    temp=head;  
    for(i=1;i<pos-1;i++)  
    {  
        temp=temp->next;  
    }  
    enode->next=temp->next;  
    enode->prev=temp;  
    temp->next=enode;  
    enode->next->prev=enode;  
}  
printf("\nAFTER INSERTING A NODE AT POSITION: ");  
display();  
return 0;  
}
```


enter n size: 4

enter node 1 data: 10

enter node 2 data: 20

enter node 3 data: 30

enter node 4 data: 40

10 20 30 40

enter newly creates node data: 50

enter position: 3

AFTER INSERTING A NODE AT POSITION: 10 20 50 30 40

```
//DELETING A NODE AT FRONT IN DOUBLY LINKED LIST
```

```
# include<stdio.h>
```

```
# include<stdlib.h>
```

```
struct node
```

```
{
```

```
    struct node *prev;
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node *head=NULL, *tail=NULL, *temp=NULL;
```

```
void create()
```

```
{
```

```
    struct node *newnode=NULL;
```

```
    int i,n;
```

```
    printf("\nenter n size: ");
```

```
    scanf("%d",&n);
```

```
    for(i=0;i<n;i++)
```

```
{
```

```
        newnode=(struct node *)malloc(sizeof(struct node));
```

```
        printf("\nenter node %d data: ",i+1);
```

```
        scanf("%d",&newnode->data);
```

```
        newnode->prev=0;
```

```
        newnode->next=0;
```

```
        if(head==NULL)
```

```
{
```

```
            head=newnode;
```

```
            tail=newnode;
```

```
}
```

```
else
{
    tail->next=newnode;
    newnode->prev=tail;
    tail=newnode;
}
}

void display()
{
    temp=head;
    while(temp!=NULL)
    {
        printf(" %d",temp->data);
        temp=temp->next;
    }
}
```

```
int main()
{
    create();
    display();
    if(head==NULL)
    {
        printf("\nLIST IS empty");
    }
    else
    {
        head=head->next;
        free(head->prev);
        head->prev=NULL;
    }
    printf("\n AFTER DELETING, THE LIST IS: ");
    display();
}
```

enter n size: 4

enter node 1 data: 20

enter node 2 data: 10

enter node 3 data: 30

enter node 4 data: 40

20 10 30 40

AFTER DELETING NODE 10 30 40

```

//DELETING A NODE AT END IN DOUBLY LINKED LIST
# include<stdio.h>
# include<stdlib.h>
struct node
{
    struct node *prev;
    int data;
    struct node *next;
};
struct node *head=NULL, *tail=NULL, *temp=NULL;
void create()
{
    struct node *newnode=NULL;
    int i,n;
    printf("\nenter n size: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        newnode=(struct node *)malloc(sizeof(struct node));
        printf("\nenter node %d data: ",i+1);
        scanf("%d",&newnode->data);
        newnode->prev=0;
        newnode->next=0;
        if(head==NULL)
        {
            head=newnode;
            tail=newnode;
        }
    }
}

```

```
    else
    {
        tail->next=newnode;
        newnode->prev=tail;
        tail=newnode;
    }
}

void display()
{
    temp=head;
    while(temp!=NULL)
    {
        printf(" %d",temp->data);
        temp=temp->next;
    }
}
```



```
int main()
{
    create();
    display();
    if(head==NULL)
    {
        printf("\nLIST IS empty");
    }
    else if(head->next==NULL)
    {
        free(head);
        head=tail=NULL;
    }
    else
    {
        tail=tail->prev;
        free(tail->next);
        tail->next=0;
    }
    printf("\nAFTER DELETING");
    display();
}
```

enter n size: 5

enter node 1 data: 10

enter node 2 data: 20

enter node 3 data: 30

enter node 4 data: 40

enter node 5 data: 50

10 20 30 40 50

AFTER DELETING 10 20 30 40

```
//DELETING A NODE AT POSITION IN DOUBLY LINKED LIST
```

```
# include<stdio.h>
# include<stdlib.h>
struct node
{
    struct node *prev;
    int data;
    struct node *next;
};
struct node *head=NULL, *tail=NULL, *temp=NULL;
void create()
{
    struct node *newnode=NULL;
    int i,n;
    printf("\nenter n size: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        newnode=(struct node *)malloc(sizeof(struct node));
        printf("\nenter node %d data: ",i+1);
        scanf("%d",&newnode->data);
        newnode->prev=0;
        newnode->next=0;
        if(head==NULL)
        {
            head=newnode;
            tail=newnode;
        }
    }
}
```

```
,  
else  
{  
    tail->next=newnode;  
    newnode->prev=tail;  
    tail=newnode;  
}  
}  
}  
void display()  
{  
    temp=head;  
    while(temp!=NULL)  
    {  
        printf(" %d",temp->data);  
        temp=temp->next;  
    }  
}
```

```

int main()
{
    create();
    display();
    int i,pos;
    printf("\nenter position: ");
    scanf("%d",&pos);
    if(head==0)
    {
        printf("\nLIST ID EMPTY");
    }
    else if(pos==1)
    {
        head=head->next;
        free(head->prev);
        head->prev=NULL;
    }
    else
    {
        temp=head;
        for(i=1;i<pos;i++)
        {
            temp=temp->next;
        }
        temp->prev->next=temp->next;
        temp->next->prev=temp->prev;
    }
    printf("\nAFTER DELETING");
    display();
    return 0;
}

```

enter n size: 4

enter node 1 data: 10

enter node 2 data: 20

enter node 3 data: 30

enter node 4 data: 40

10 20 30 40

enter position: 2

AFTER DELETING 10 30 40

```

//REVERSING A LIST
# include<stdio.h>
# include<stdlib.h>
struct node
{
    struct node *prev;
    int data;
    struct node *next;
};
struct node *head=0,*temp=0,*tail=0;
void create()
{
    struct node *newnode=0;
    int i,n;
    printf("\nenter n size: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        newnode=(struct node *)malloc(sizeof(struct node));
        printf("\nenter node %d data: ",i+1);
        scanf("%d",&newnode->data);
        newnode->prev=NULL;
        newnode->next=NULL;
        if(head==NULL)
        {
            head=tail=newnode;
        }
    }
}

```



```
else
{
    tail->next=newnode;
    newnode->prev=tail;
    tail=newnode;
}
}

void display()
{
    temp=head;
    while(temp!=0)
    {
        printf(" %d",temp->data);
        temp=temp->next;
    }
}
```

```
int main()
{
    create();
    display();
    struct node *currentnode=0, *nextnode=0;
    currentnode=head;
    while(currentnode!=NULL)
    {
        nextnode=currentnode->next;
        currentnode->next=currentnode->prev;
        currentnode->prev=nextnode;
        currentnode=nextnode;
    }
    currentnode=head;
    head=tail;
    tail=currentnode;
    printf("\nAFTER REVERSING ");
    display();
}
```

enter n size: 5

enter node 1 data: 10

enter node 2 data: 20

enter node 3 data: 30

enter node 4 data: 40

enter node 5 data: 50

10 20 30 40 50

AFTER REVERSING 50 40 30 20 10